



UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA

Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Laboratorio · Autenticación con PASETO · LAB PASETO V1, V2 y V3

ESPACIO ACADÉMICO:

Seguridad en el Desarrollo de Software

PRESENTADO A:

Sergio Arley Puerto Moreno

AUTOR:

Juan Camilo Calderón Delgado

2026

1. El problema: demostrar quién eres sin repetir la contraseña

Cuando un usuario inicia sesión en una API, el servidor lo reconoce en ese momento, pero en la siguiente petición ya no sabe quién es. Si la solución fuera mandar el correo y la contraseña en cada llamada, la contraseña viajaría todo el tiempo, y eso es justo lo que no queremos. La alternativa es que el login entregue un token y que las peticiones siguientes lo presenten en lugar de la contraseña. En estos tres laboratorios hice eso con PASETO (Platform-Agnostic Security Tokens), sobre una API de hospital hecha en Node.js y Express.

El sello del notario

Para entender `v4.public` me ayudó pensar en un notario. Solo el notario tiene el sello con el que firma los documentos, pero cualquiera puede ir a la notaría y comprobar que una firma es auténtica. Aquí la clave privada es el sello y solo la tiene el servidor, que firma el token con `v4.Sign`. La clave pública sirve para comprobarlo con `v4.Verify`, y si alguien cambia un solo carácter del token, la firma deja de coincidir.

Que el propósito sea `public` no significa que el token sea público, significa que se usa criptografía asimétrica. Lo que sí hay que tener claro es que el contenido del token se puede leer, lo que nadie puede hacer es modificarlo sin que se note. Por eso dentro solo van `sub`, `email` y `rol`, junto con `iat` y `exp`, y nunca la contraseña.

2. Entorno y repositorio

Las tres versiones usan Node.js con ES Modules, Express 5, el paquete `paseto` y `dotenv`, con `nodemon` solo para desarrollo. La V2 agrega `swagger-jsdoc` y `swagger-ui-express`, y la V3 agrega `express-validator`. Los usuarios están en memoria y hay tres roles: administrador, médico y paciente.

El código de cada versión y su README están en el repositorio de GitHub:

<https://github.com/JuanK2550/lab-paseto-v1-v2-v3>.

Las pruebas de la V1 las hice con Postman en el puerto 3001, que dejé configurado en el `.env`. Las de la V2 y la V3 las hice desde Swagger UI en `http://localhost:3000/api-docs`, y cada captura muestra el `curl` generado, la URL, el código HTTP y la respuesta.

3. Versión 1: login y ruta protegida

La primera versión tiene solo dos rutas, `POST /api/auth/login` y `GET /api/auth/perfil`. El login compara el correo y la contraseña con un único usuario escrito en el código y, si coinciden, firma con `v4.Sign` un token con los claims `sub`, `email` y `rol`. El middleware `autenticarPaseto` lee el encabezado `Authorization`, separa el esquema Bearer del token y lo verifica con la clave pública. Si es válido guarda los claims en `req.usuario` y deja pasar la petición.

Las claves se generan cada vez que arranca el servidor, así que al reiniciarlo los tokens anteriores dejan de servir. En la prueba 1 el login responde 200 con un token que empieza por `v4.public.`, y el servidor también lo imprime en la consola junto con el aviso de que las claves fueron generadas.

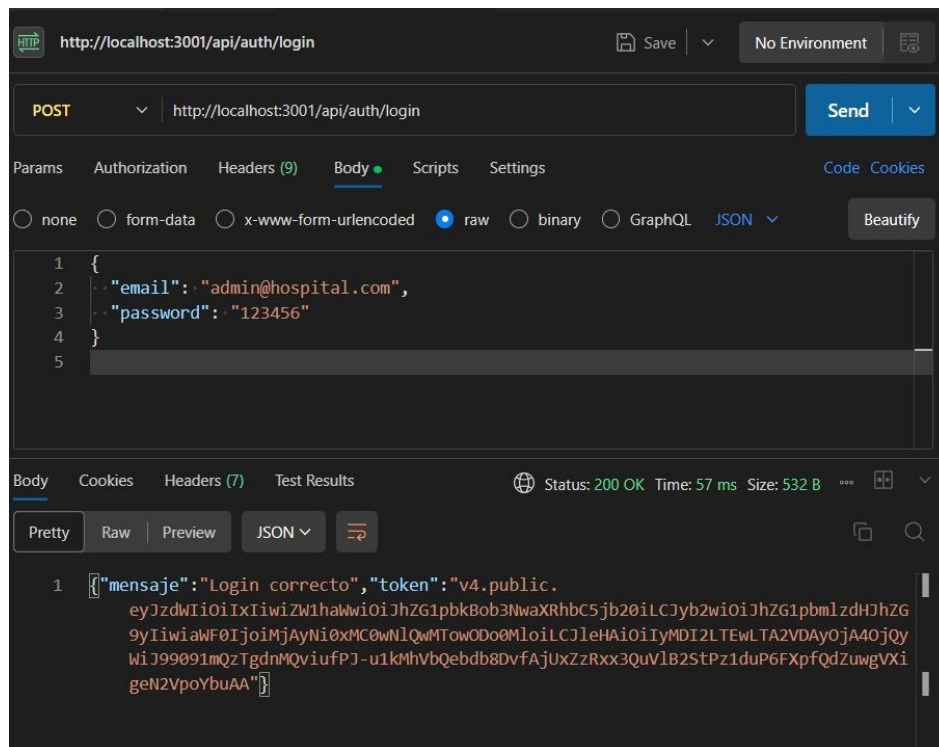


Figura 1. Prueba 1. Login correcto: 200 y token `v4.public.`

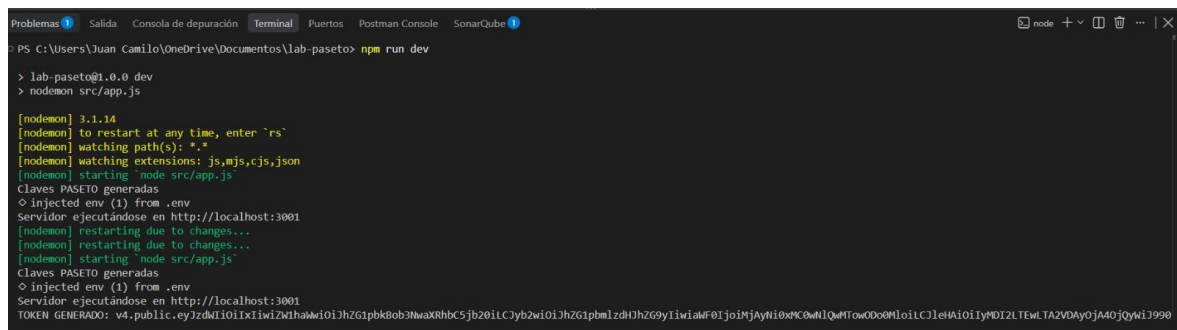


Figura 2. Consola del servidor: claves PASETO generadas y token impreso.

Casos de rechazo

En la prueba 2 mando una contraseña incorrecta y la API responde 401 con un mensaje genérico. En la prueba 3 pido el perfil sin token y también responde 401, lo que demuestra que la ruta está protegida.

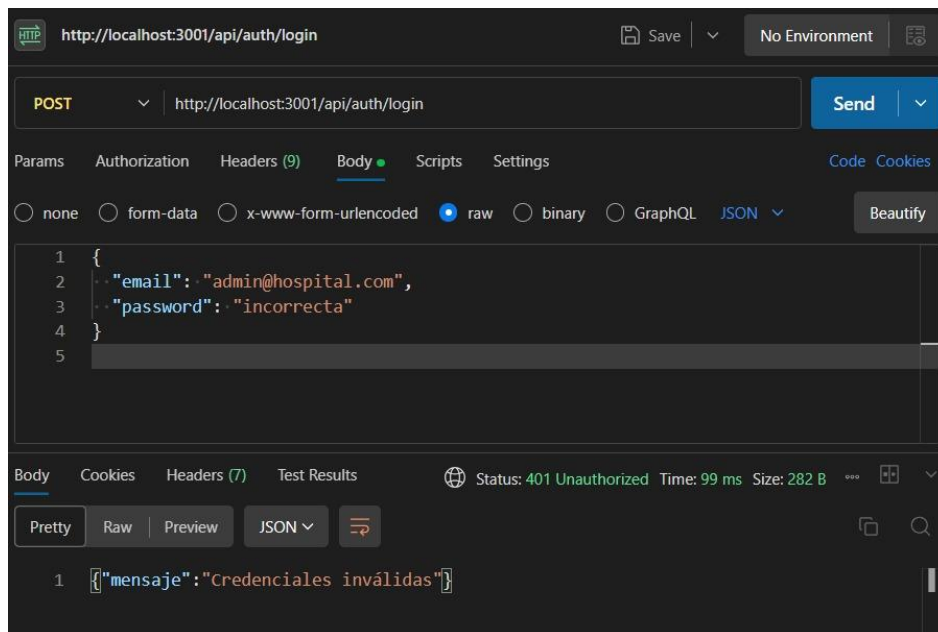


Figura 3. Prueba 2. Contraseña incorrecta: 401, credenciales inválidas.

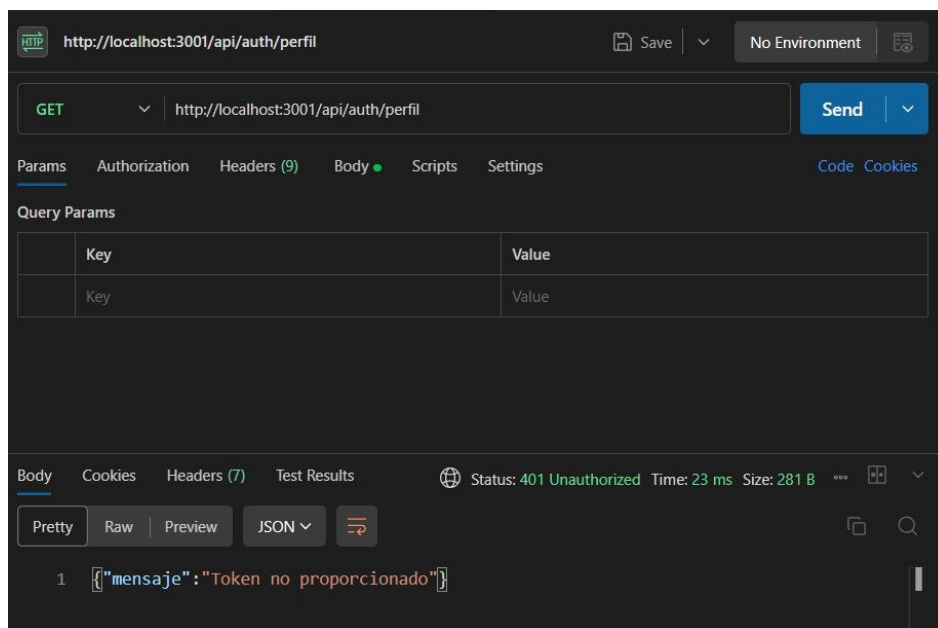


Figura 4. Prueba 3. Perfil sin token: 401, token no proporcionado.

Token válido y token alterado

En la prueba 4 pongo el token como Bearer Token en Postman y el perfil responde 200 con los claims, entre ellos `iat` y `exp`, que están separados por una hora. En la prueba 5 cambio un carácter del token y la verificación falla con 401, porque la firma ya no coincide con el contenido.

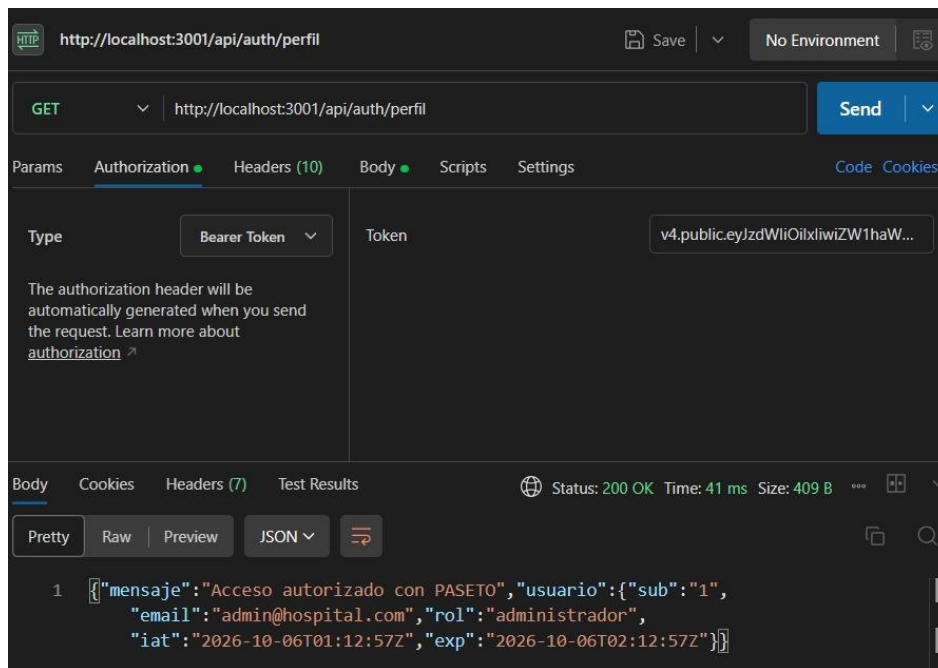


Figura 5. Prueba 4. Perfil con token válido: 200 y claims del usuario.

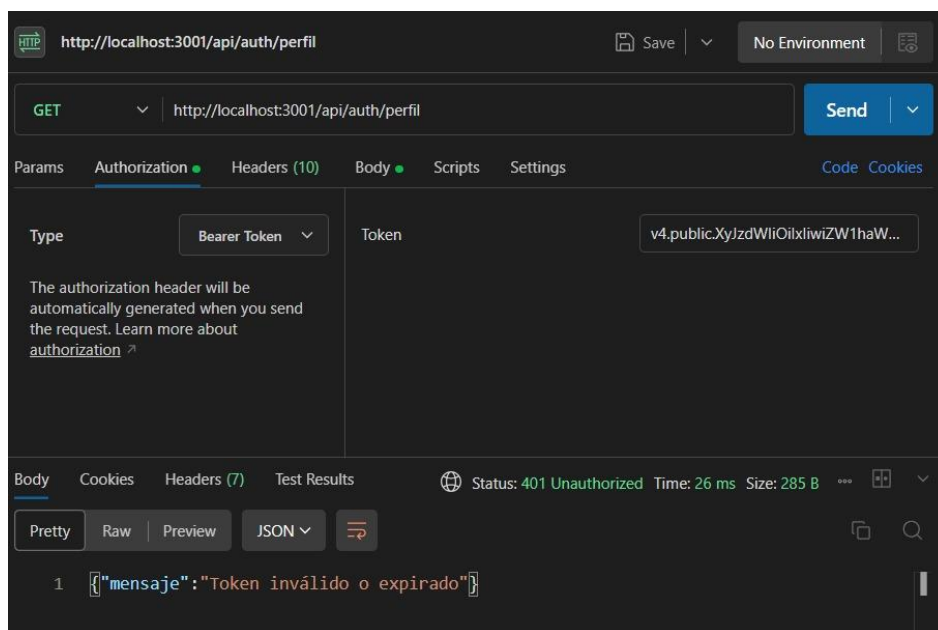


Figura 6. Prueba 5. Perfil con token alterado: 401, token inválido o expirado.

4. Versión 2: usuarios, roles y Swagger

En la segunda versión reemplacé el usuario único por tres usuarios locales con roles distintos. El login busca al usuario por correo, revisa la contraseña y que esté activo, y responde el mismo 401 tanto si falla el correo como si falla la contraseña, para no revelar qué correos existen. Si el usuario está deshabilitado responde 403.

Agregué el middleware `autorizarRoles`, que lee el rol de `req.usuario` y devuelve 403 si no está permitido. Con esto quedó clara la diferencia entre los dos códigos: 401 es que no se pudo comprobar quién eres, y 403 es que sí se sabe, pero no tienes permiso. También documenté la API con Swagger y un esquema Bearer, para poder autorizar los endpoints desde el navegador.

Administrador: login, perfil y listado

En las pruebas 1 y 2 inicio sesión como administrador y consulto el perfil, que responde 200 sin la contraseña. En la prueba 3 pido el perfil sin token y responde 401. En la prueba 4 el administrador lista los usuarios y también obtiene 200.

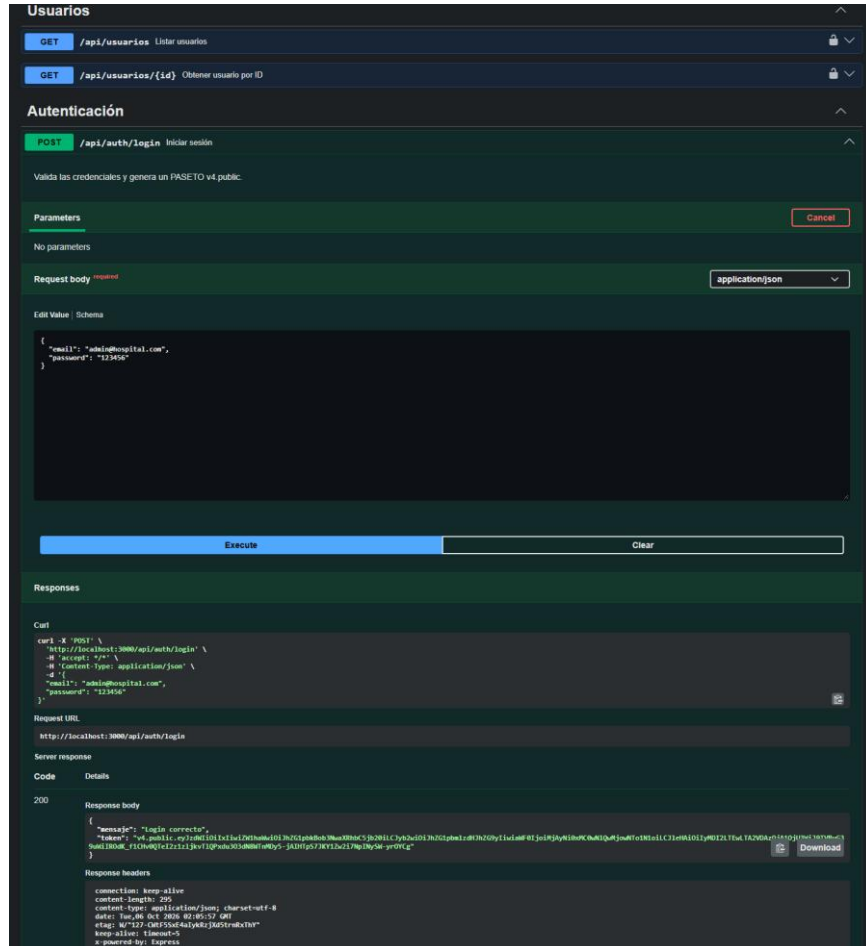


Figura 7. Prueba 1. Login del administrador: 200 y token.

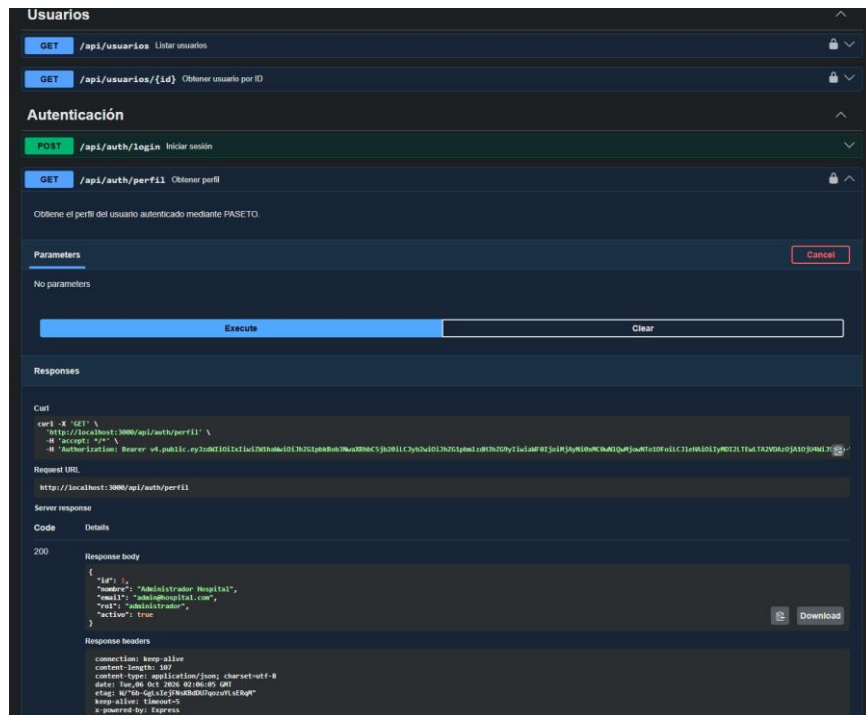


Figura 8. Prueba 2. Perfil del administrador: 200.

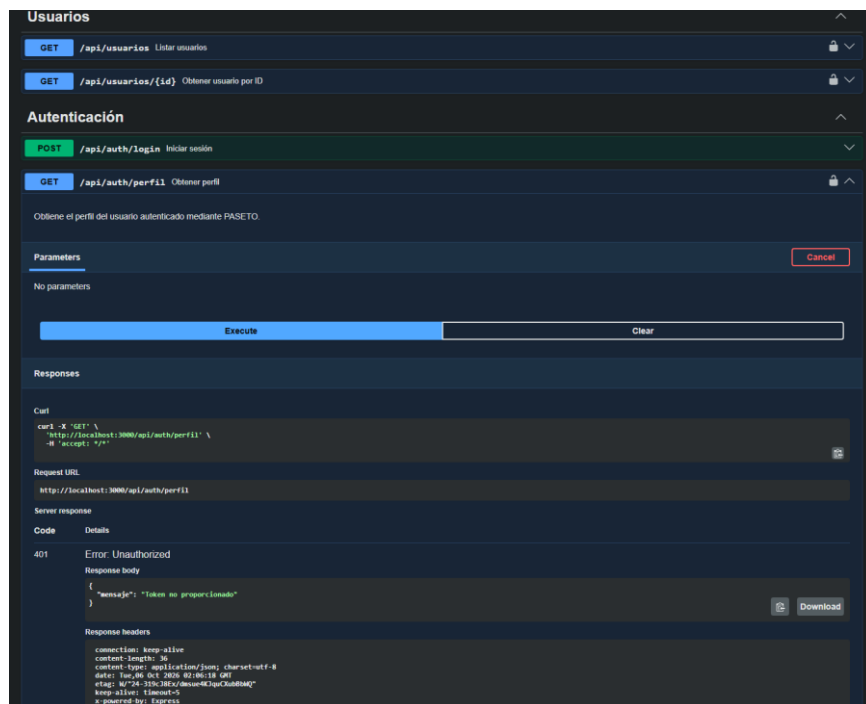


Figura 9. Prueba 3. Perfil sin token: 401.

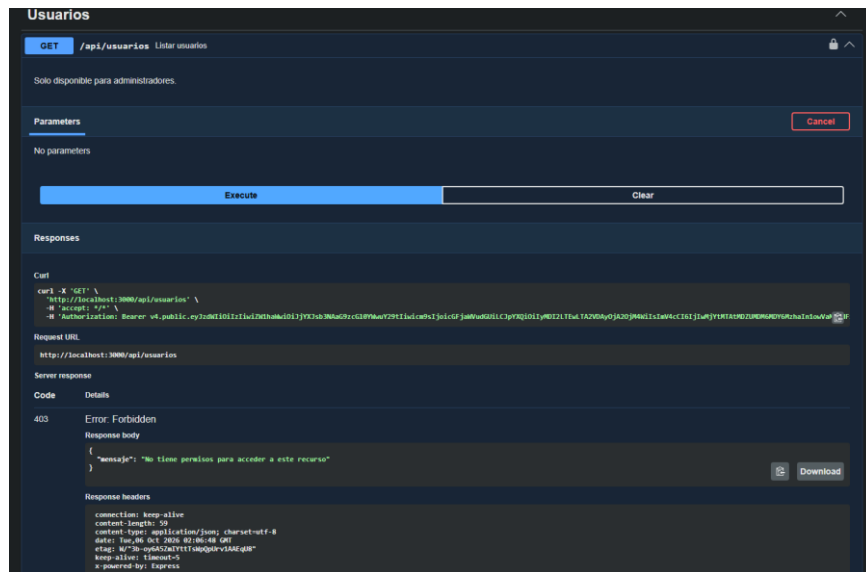


Figura 14. Prueba 8. Paciente listando usuarios: 403.

Credenciales, token alterado y usuario inexistente

Con una contraseña incorrecta (prueba 9) y con un correo que no existe (prueba 10) el login responde 401 con el mismo mensaje. Un token manipulado también responde 401 (prueba 11), y pedir un usuario que no existe como administrador devuelve 404 (prueba 12).

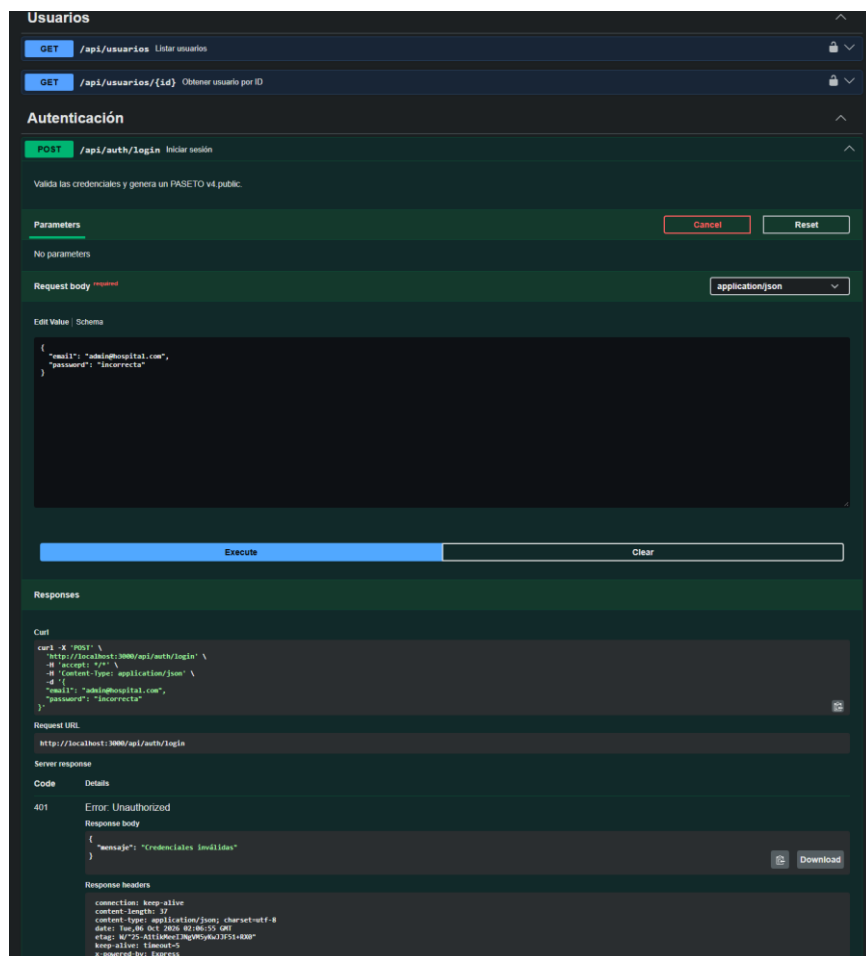
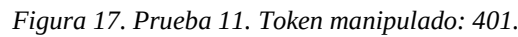
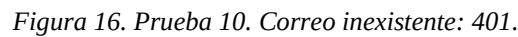


Figura 15. Prueba 9. Contraseña incorrecta: 401.



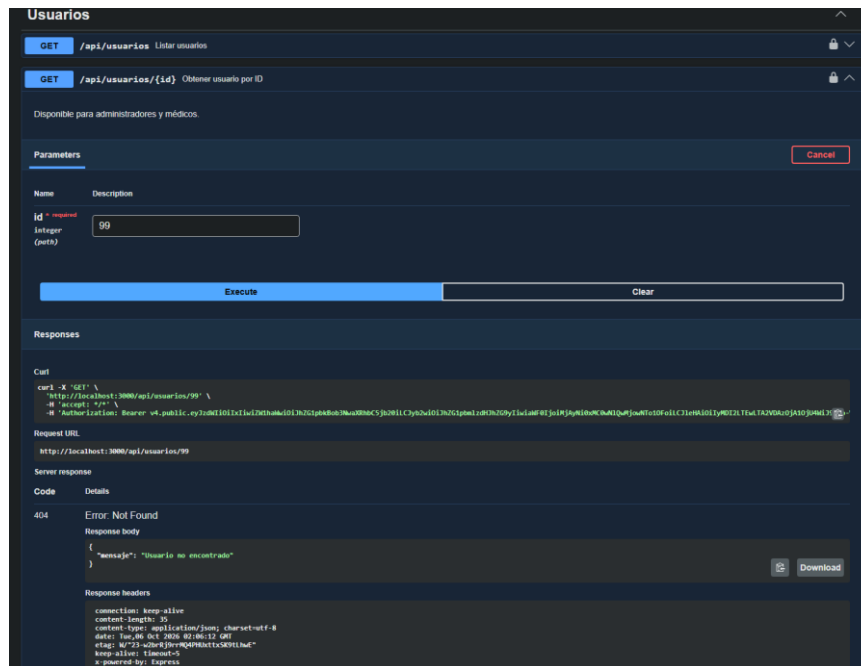


Figura 18. Prueba 12. Usuario 99 inexistente: 404.

5. Análisis de dependencias con npm audit

En la V2 también corrí `npm audit`, que reportó 3 vulnerabilidades altas en el paquete `braces`. Ese paquete no lo instalé yo, llega por `nodemon` y `chokidar`, y `nodemon` está en `devDependencies`, así que es una herramienta de desarrollo. Al correr `npm audit --omit=dev` el resultado fue 0 vulnerabilidades en las dependencias de producción.

No ejecuté `npm audit fix --force`, porque proponía instalar una versión muy antigua de `nodemon` con cambios incompatibles. Preferí identificar el paquete, ver si era una dependencia directa o transitiva, si era de desarrollo o de producción, y documentar el hallazgo en lugar de forzar una corrección. En la V3 el resultado fue el mismo.

6. Versión 3: CRUD completo con validaciones

La tercera versión completa el recurso de usuarios con `GET`, `POST`, `PUT`, `PATCH` y `DELETE`. Cada ruta pasa por cuatro etapas en un orden fijo: autenticación con el `PASETO`, autorización por rol, validación del cuerpo con `express-validator` y, al final, el controlador. El orden importa porque no tiene sentido validar los datos de alguien que ni siquiera tiene permiso.

De la validación saco los datos con `matchedData()`, para que el controlador use solo los campos que declaré y no todo lo que mande el cliente. La diferencia entre los dos métodos de actualización es que `PUT` reemplaza el usuario completo y exige todos los campos, mientras que `PATCH` cambia solo lo que se envía. Además uso 201 al crear, 204 al eliminar, 400 para datos inválidos, 404 si el usuario no existe y 409 si el correo ya está registrado. Los permisos quedaron así: el administrador puede hacer todas las operaciones, el médico solo puede consultar un usuario por ID, y el paciente solo puede ver su propio perfil. Las contraseñas nunca se devuelven en las respuestas.

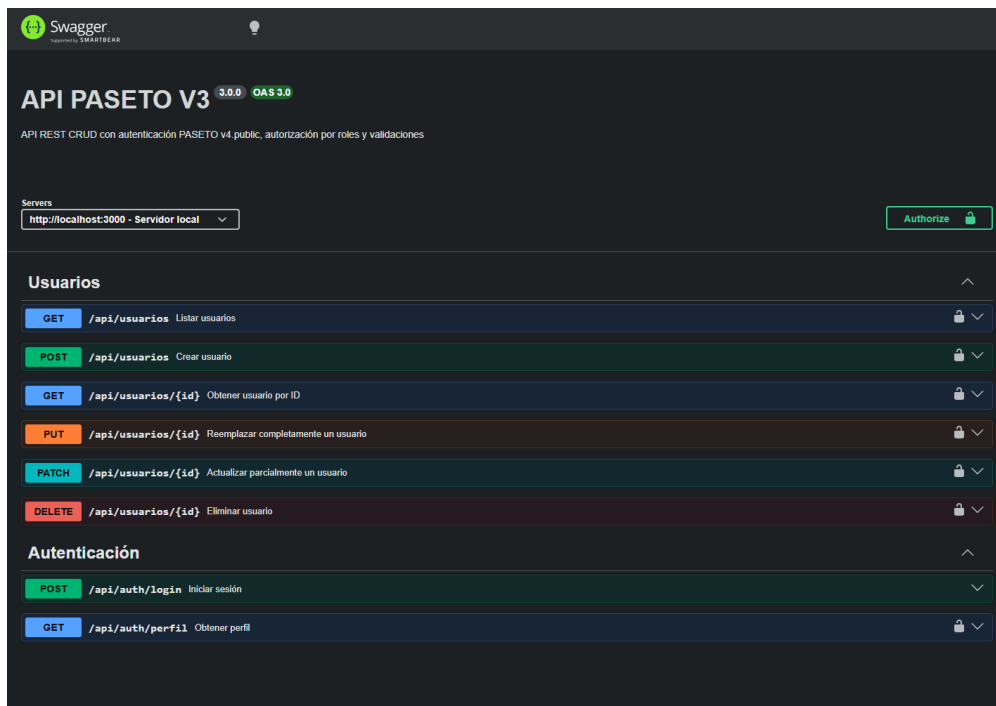


Figura 19. Swagger UI de la V3 con sus ocho endpoints.

7. Pruebas de la V3

Las 23 pruebas del README las ejecuté desde Swagger UI, más una prueba adicional con un usuario deshabilitado. En todas el código obtenido fue el esperado.

Login, autorización y perfil

En la prueba 1 el administrador inicia sesión y obtiene el token, que cargo en el botón Authorize. Con él, el perfil de la prueba 2 responde 200.

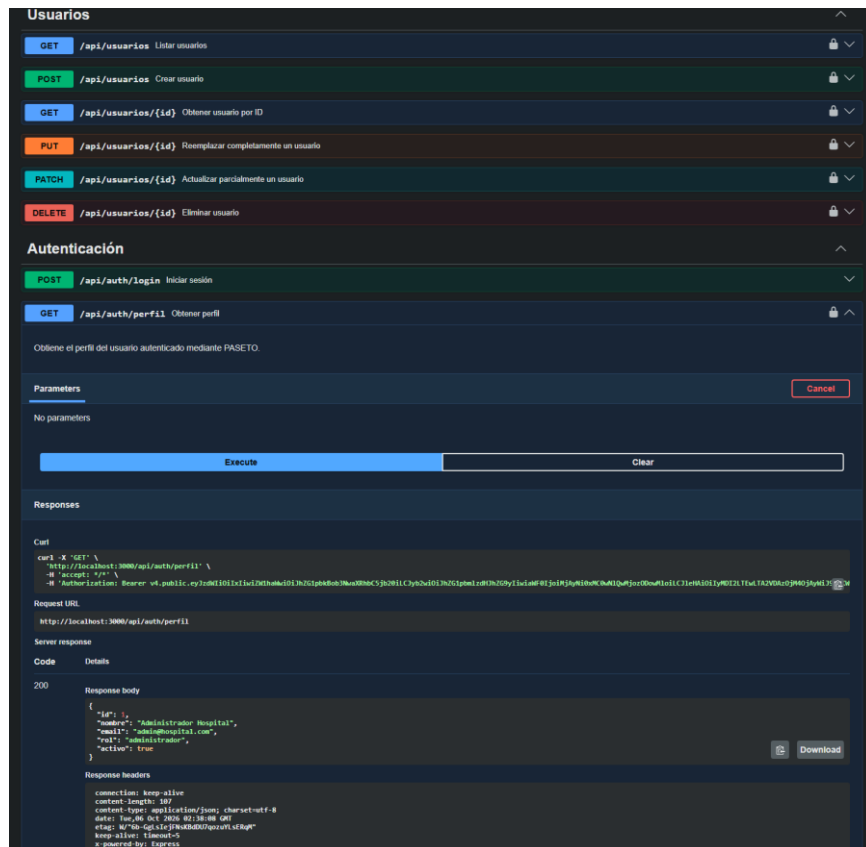


Figura 22. Prueba 2. Perfil con token válido: 200.

Lectura y creación

La prueba 3 lista los usuarios sin mostrar las contraseñas. En la prueba 4 creo a Ana Torres y la API responde 201, y en la prueba 5 la consulto por su ID para comprobar que quedó guardada.

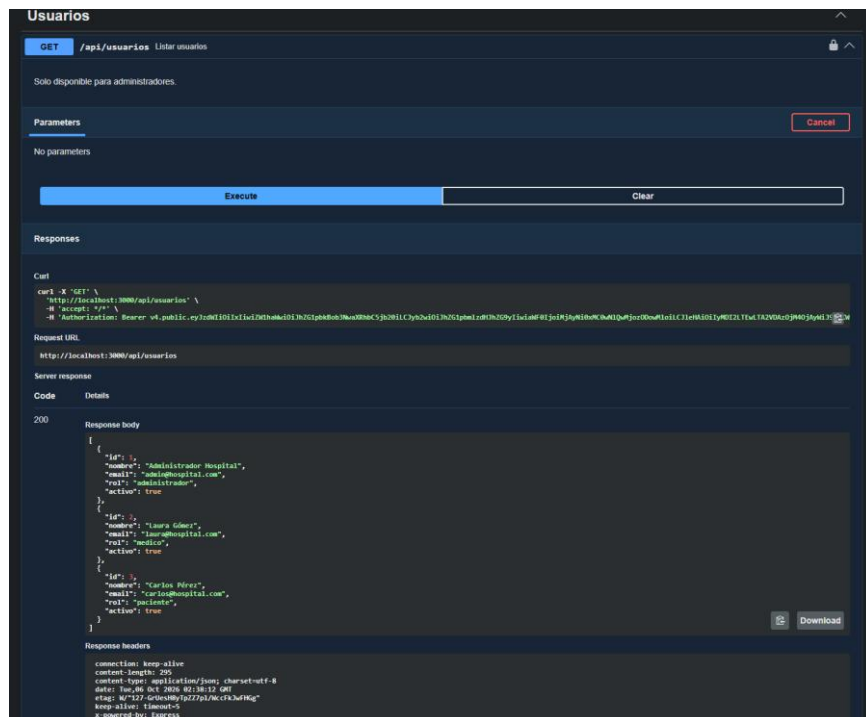


Figura 23. Prueba 3. Listar usuarios sin contraseñas: 200.

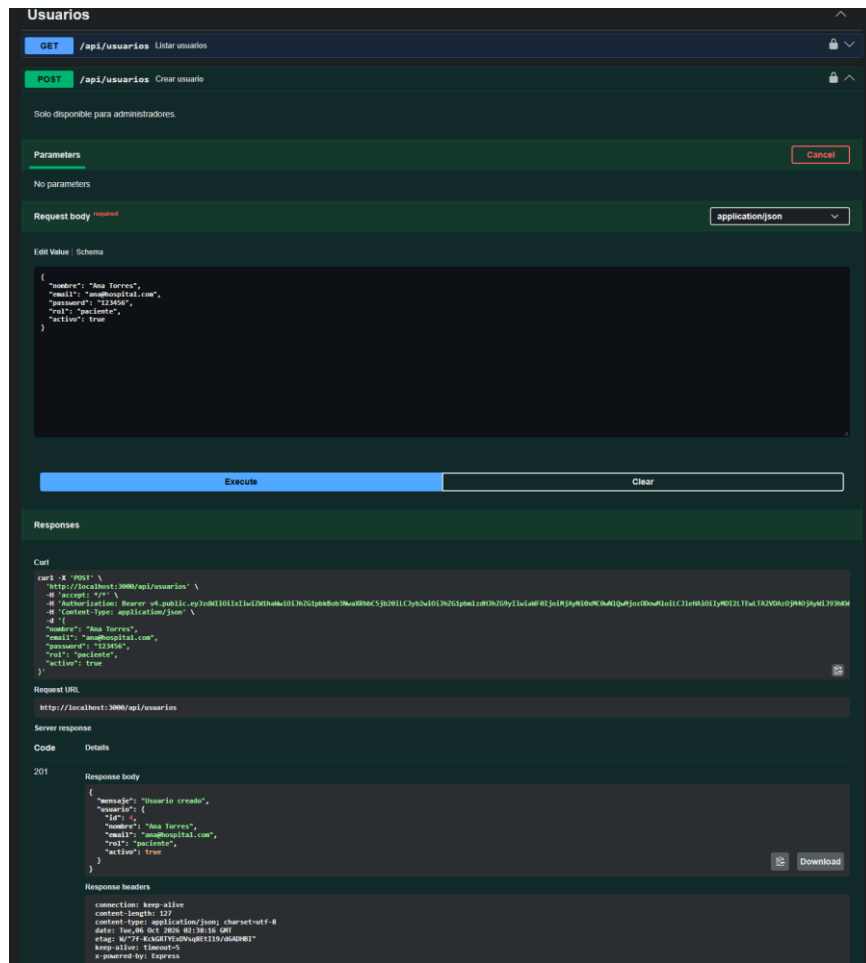


Figura 24. Prueba 4. Crear usuario: 201 Created.

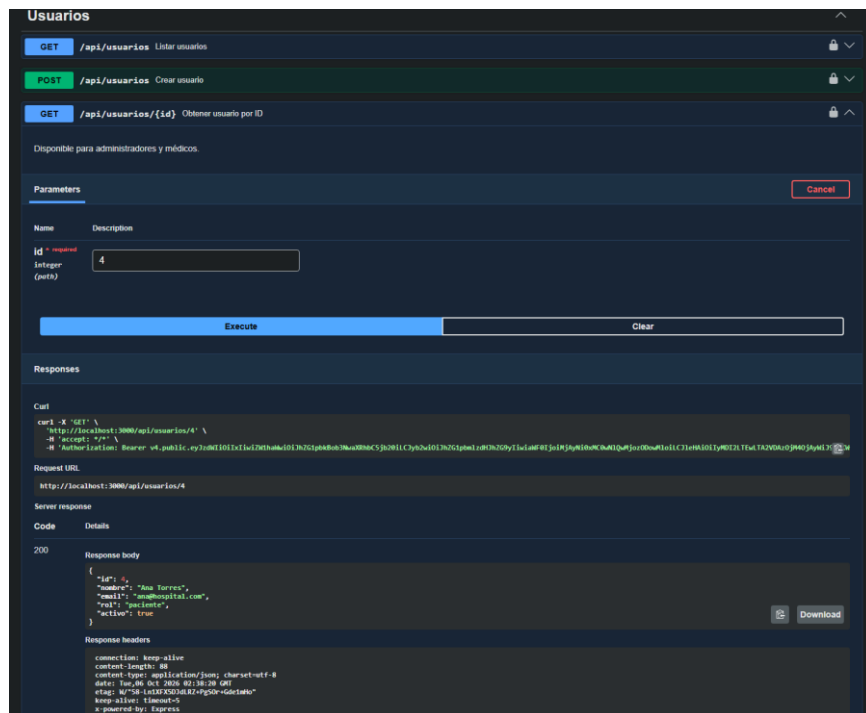


Figura 25. Prueba 5. Consultar el usuario creado: 200.

Actualización con PUT y PATCH

En la prueba 6 el **PUT** reemplaza al usuario completo y responde 200. En la prueba 7 el **PUT** solo con el nombre falla con 400, porque faltan los demás campos. En la prueba 8 el **PATCH** cambia solo **activo** y el resto de los datos queda igual, y en la prueba 9 un **PATCH** con el cuerpo vacío responde 400.

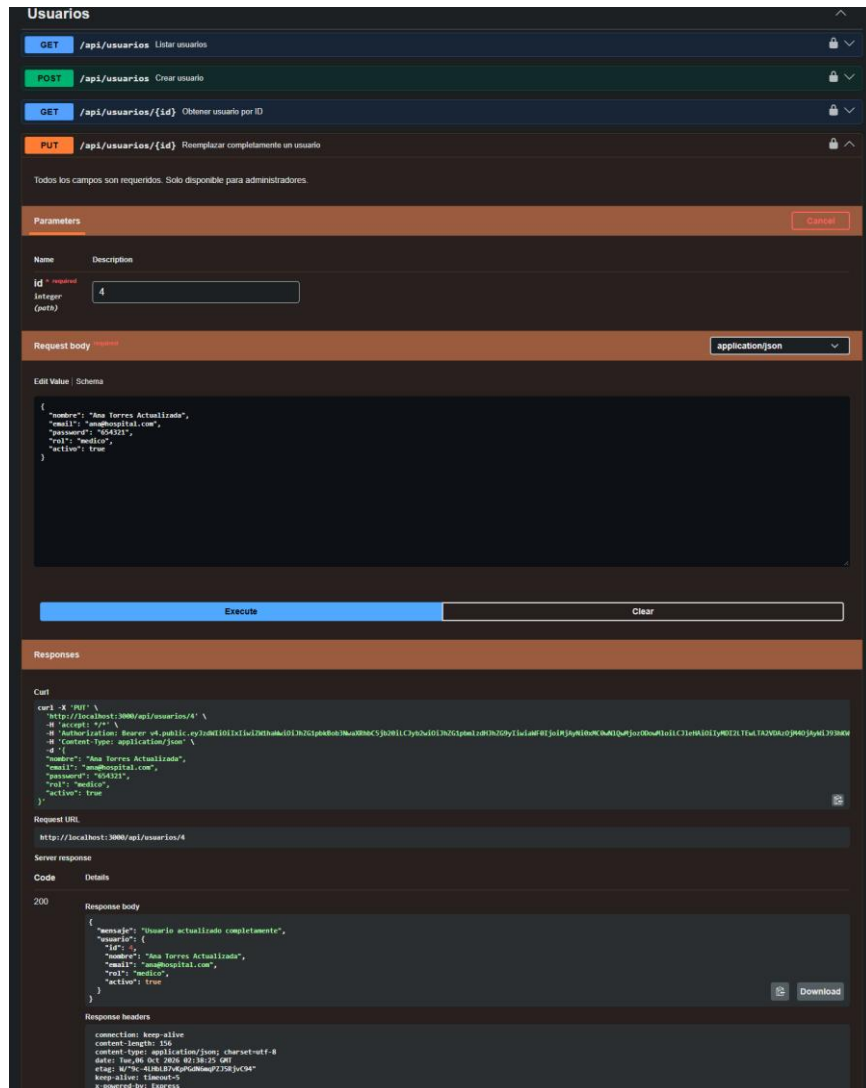


Figura 26. Prueba 6. PUT completo: 200.

Figura 27. Prueba 7. PUT incompleto: 400, datos inválidos.

Figura 27. Prueba 7. PUT incompleto: 400, datos inválidos.

Usuarios

- GET /api/usuarios Listar usuarios
- POST /api/usuarios Crear usuario
- GET /api/usuarios/{id} Obtener usuario por ID
- PUT /api/usuarios/{id} Reemplazar completamente un usuario
- PATCH /api/usuarios/{id} Actualizar parcialmente un usuario

Permite modificar uno o varios campos. Solo disponible para administradores.

Parameters Cancel Reset

Name	Description
id required	
Integer (ints)	4

Request body required application/json

Edit Value Schema

```
{
  "active": false
}
```

Execute Clear

Responses

Curl

```
curl -X 'PATCH' \
  http://localhost:3000/api/usuarios/4' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer <token>' \
  -H 'Content-Type: application/json' \
  -d '{
    "active": false
  }'
```

Request URL

http://localhost:3000/api/usuarios/4

Server response

Code	Details
200	Response body <pre>{ "message": "Usuario actualizado parcialmente", "response": { "id": 4, "name": "Ana Torres Actualizada", "email": "ana@hospital.com", "role": "medica", "active": false } }</pre> Response headers <pre>connection: keep-alive content-length: 156 content-type: application/json; charset=utf-8 date: Tue, 06 Oct 2020 02:38:34 GMT etag: W/"9c-p1wZJd8Qmaw7Pp10Bk19akQ" keep-alive: timeout=5 powered-by: Express</pre> Download

Figura 28. Prueba 8. PATCH parcial: 200.

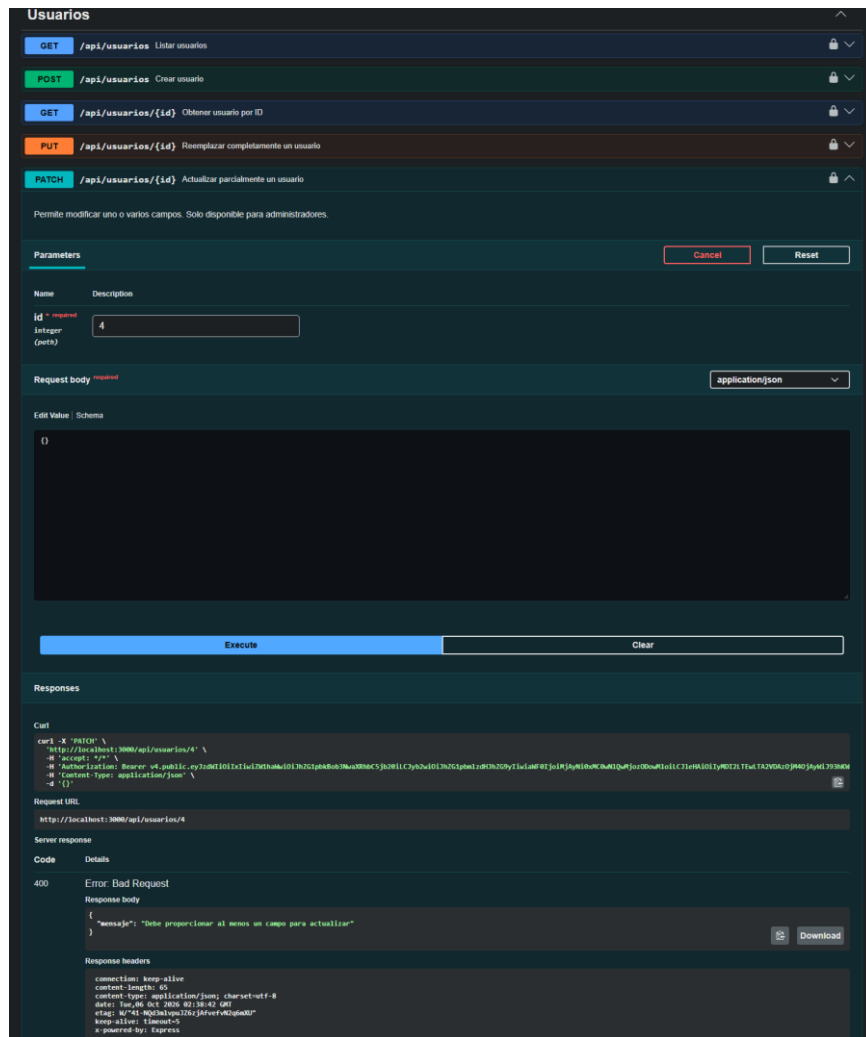


Figura 29. Prueba 9. PATCH con cuerpo vacío: 400.

Validaciones y conflicto

Un correo sin formato válido responde 400 (prueba 10) y un rol que no existe, como **superadministrador**, también (prueba 11). Un correo que ya está registrado responde 409 (prueba 12), porque la petición es válida pero choca con los datos que ya existen.

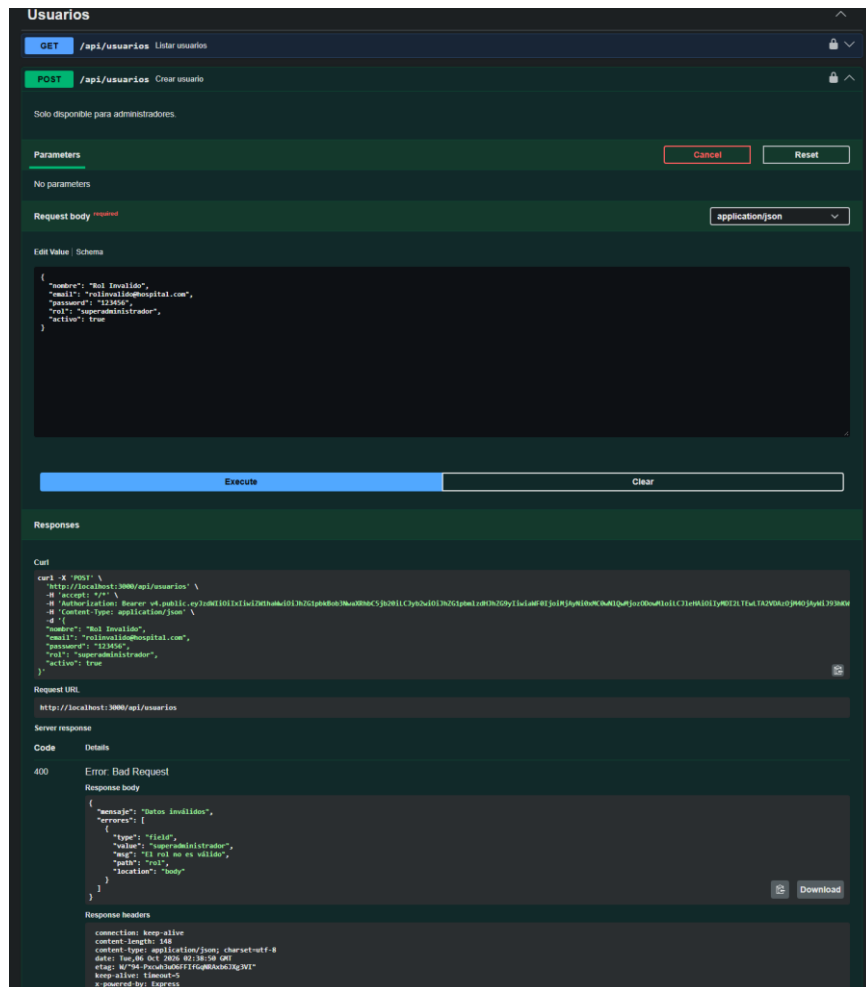


Figura 31. Prueba 11. Rol inválido: 400.

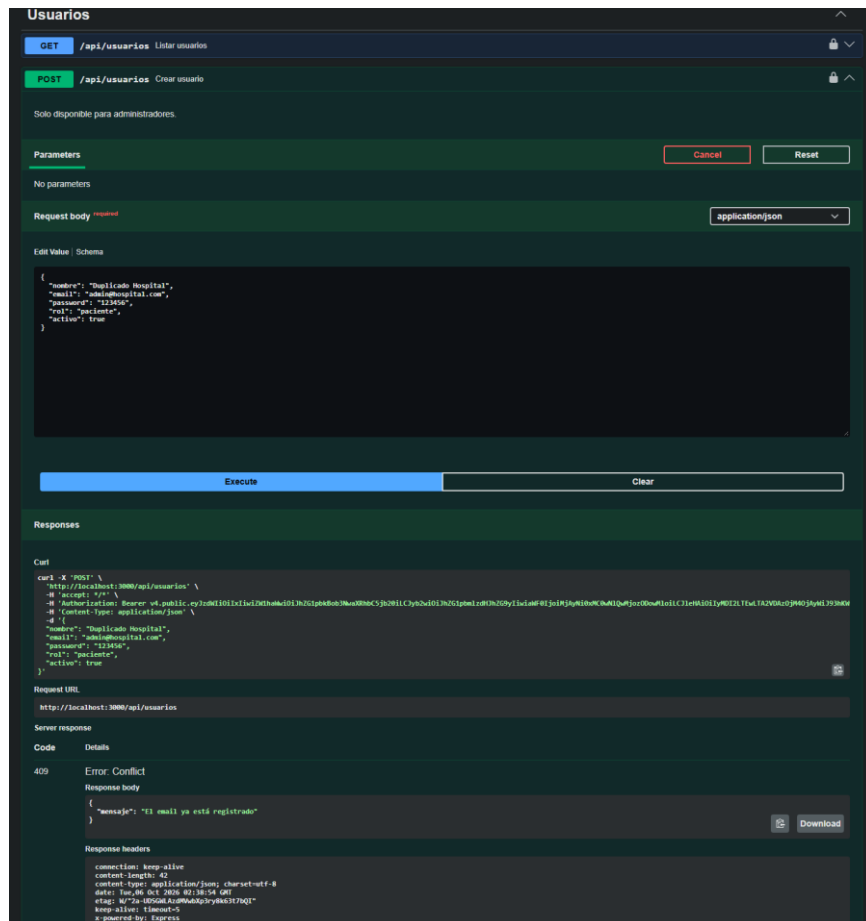


Figura 32. Prueba 12. Correo ya registrado: 409 Conflict.

Usuario inexistente y eliminación

Pedir el usuario 999 responde 404 (prueba 13). El **DELETE** del usuario 4 responde 204 sin cuerpo (prueba 14), y al consultarlo de nuevo la respuesta es 404, lo que confirma que se eliminó (prueba 15).

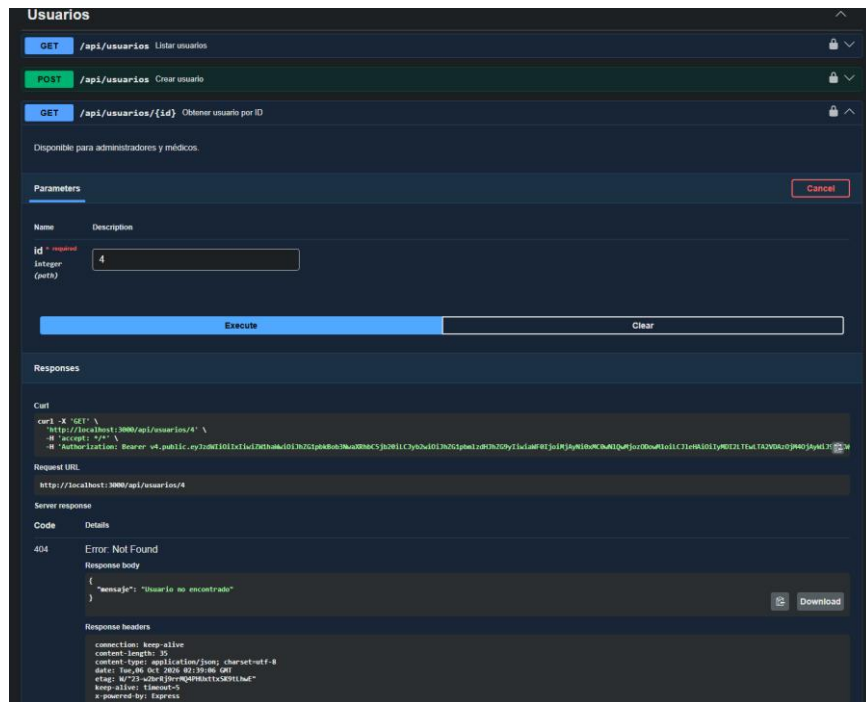


Figura 35. Prueba 15. Consulta tras el DELETE: 404.

Token ausente o alterado

Sin token, la ruta de usuarios responde 401 (prueba 16). Con un carácter del token cambiado también responde 401 (prueba 17), porque la verificación criptográfica falla.

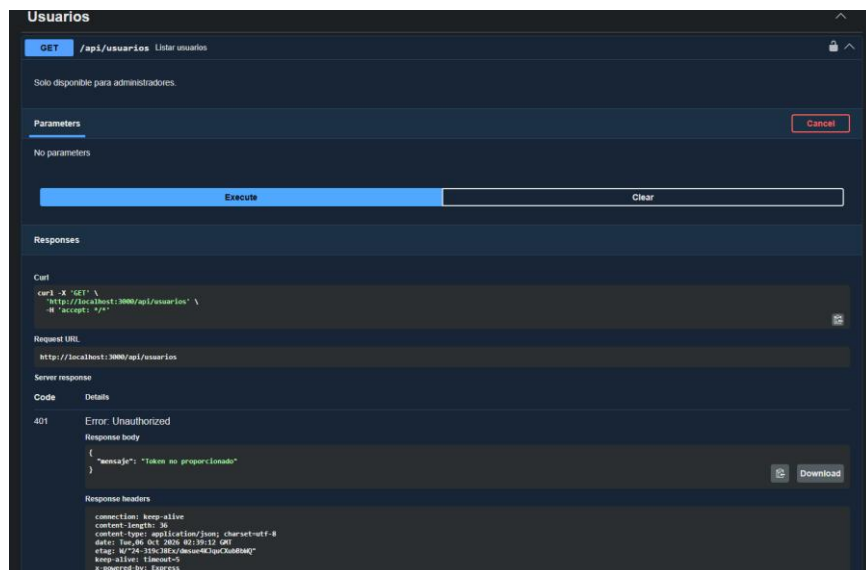


Figura 36. Prueba 16. Listar usuarios sin token: 401.

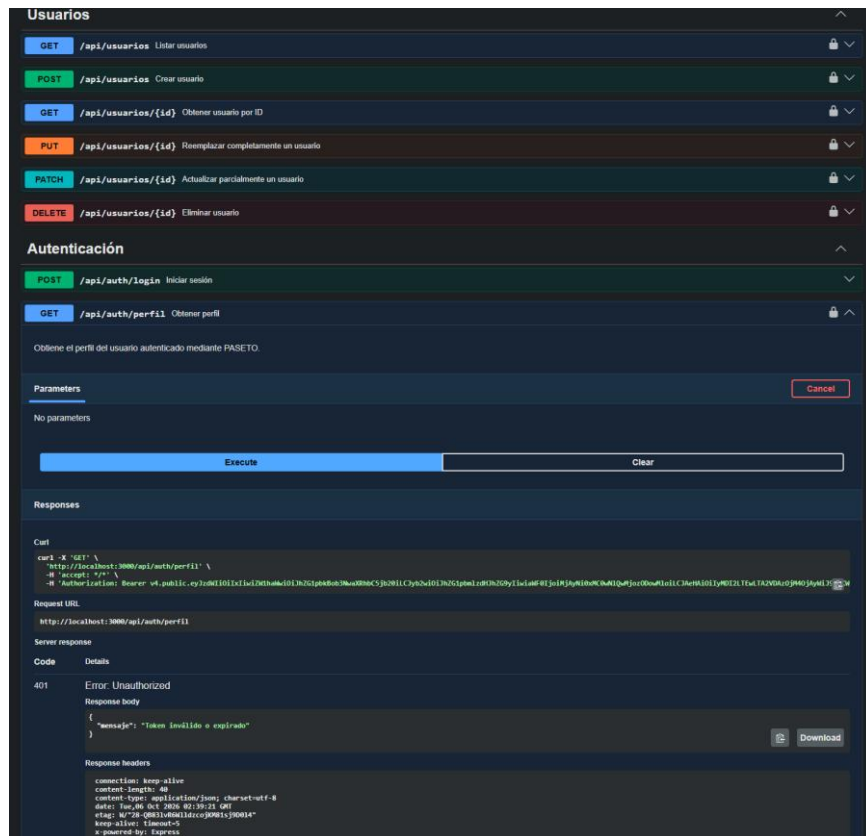


Figura 37. Prueba 17. Token manipulado: 401.

Rol médico

Laura puede consultar el usuario 1 (prueba 18), pero no puede listar usuarios (prueba 19) ni crear uno (prueba 20). La prueba 20 es la más útil de todas: mando un cuerpo totalmente válido y aun así recibo 403 y no 400, lo que confirma que la autorización se evalúa antes que la validación y que el controlador nunca se ejecuta.

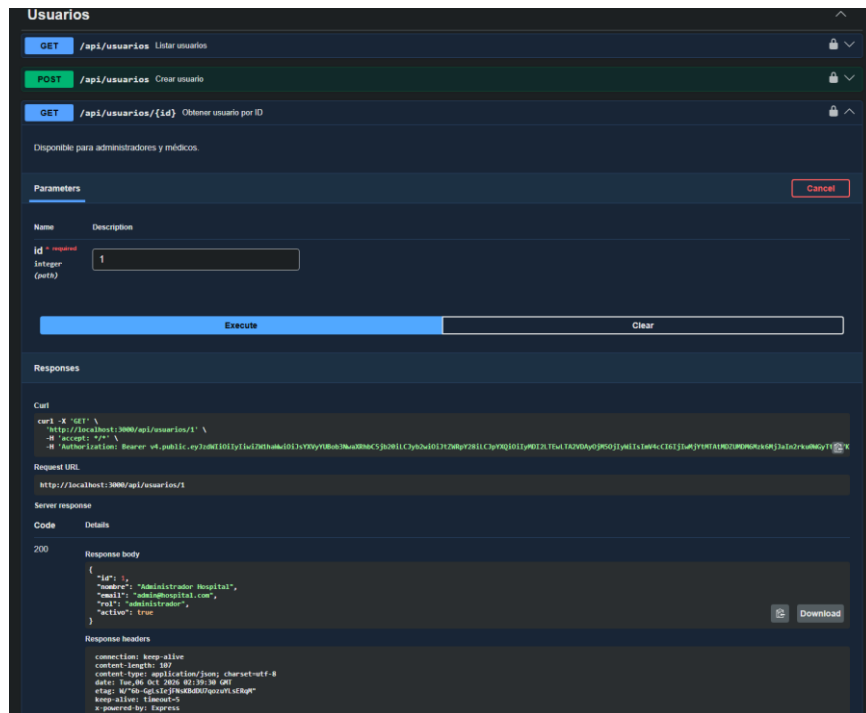


Figura 38. Prueba 18. Médico consultando el usuario 1: 200.

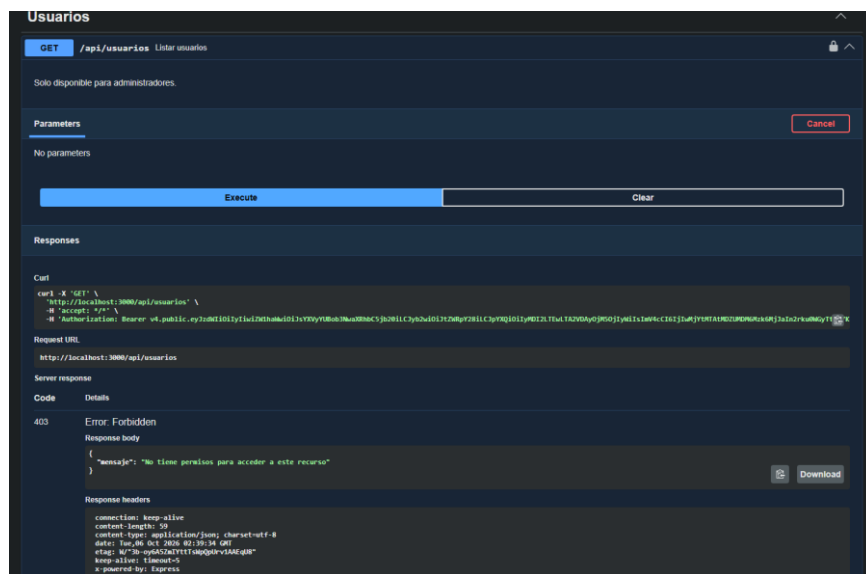


Figura 39. Prueba 19. Médico listando usuarios: 403.

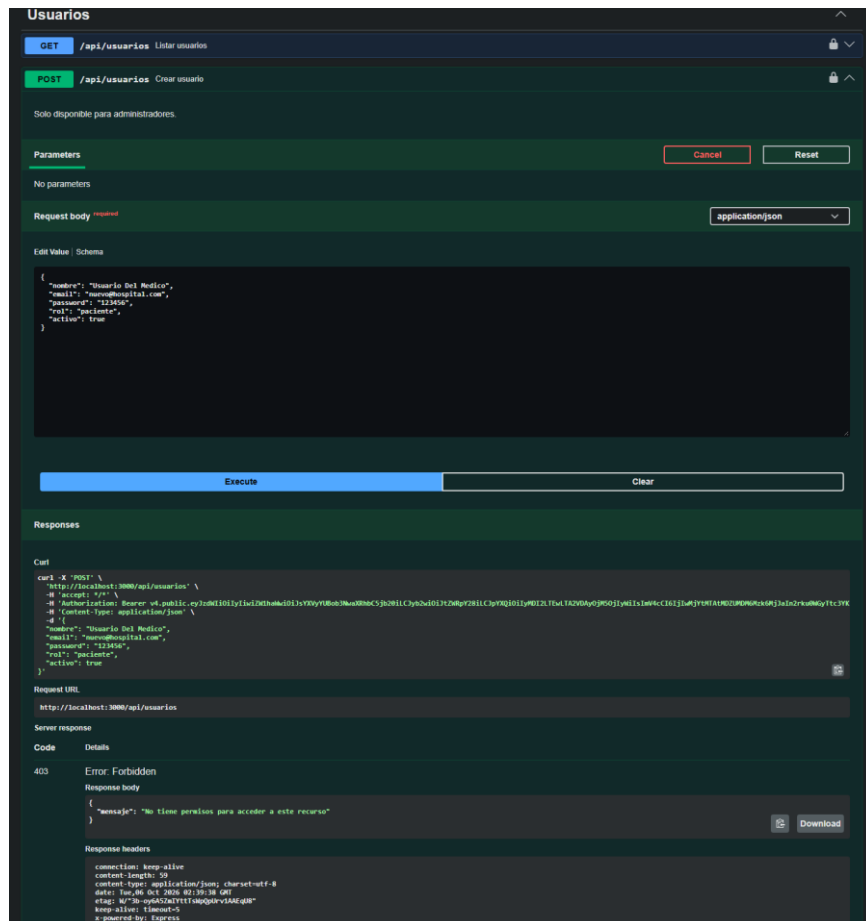


Figura 40. Prueba 20. Médico creando con cuerpo válido: 403.

Rol paciente

Carlos puede ver su perfil (prueba 21), pero recibe 403 al listar usuarios y también al consultar el usuario 1.

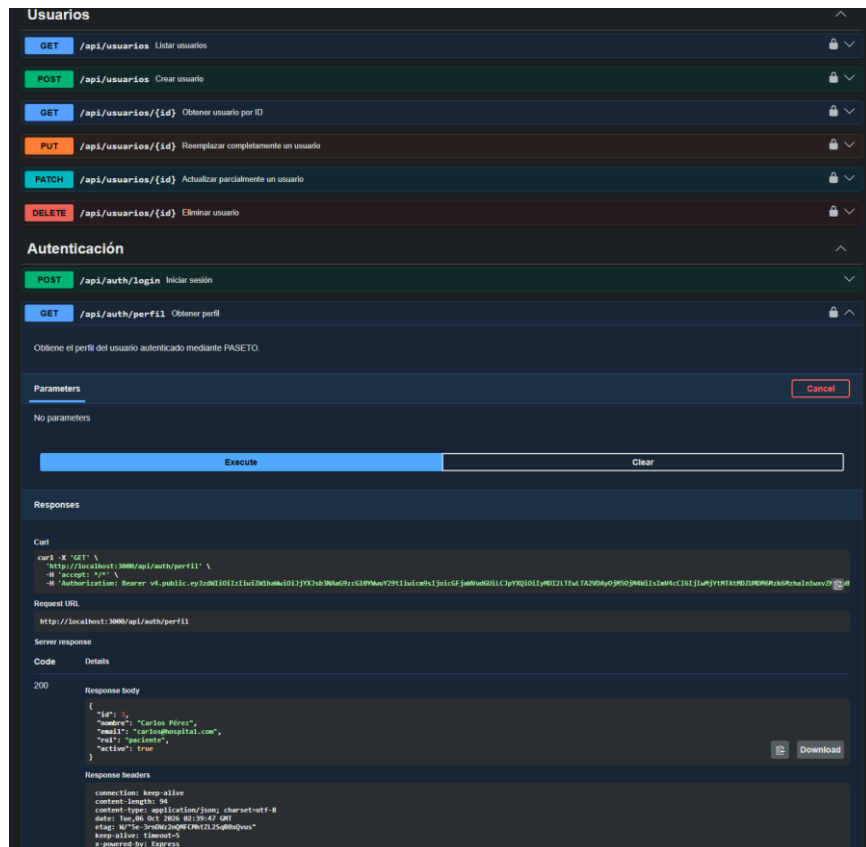


Figura 41. Prueba 21. Perfil del paciente: 200.

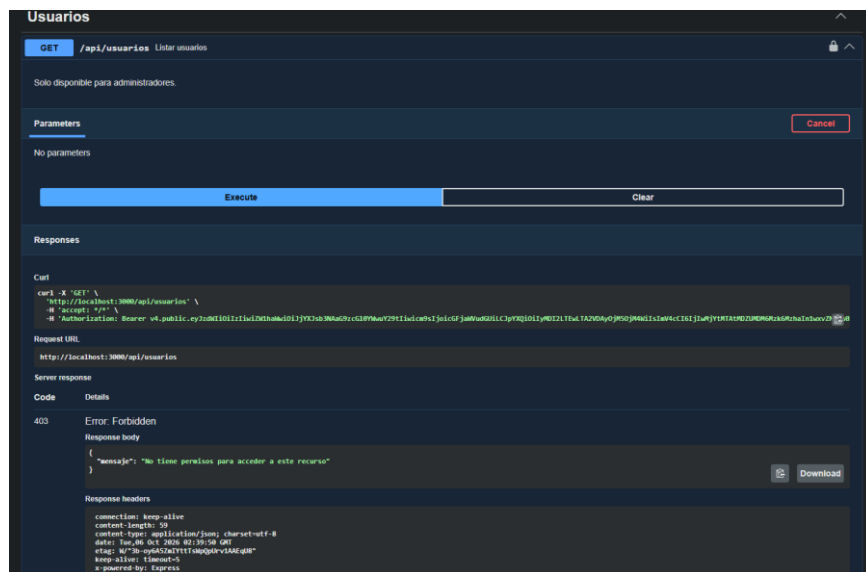


Figura 42. Prueba 21b. Paciente listando usuarios: 403.

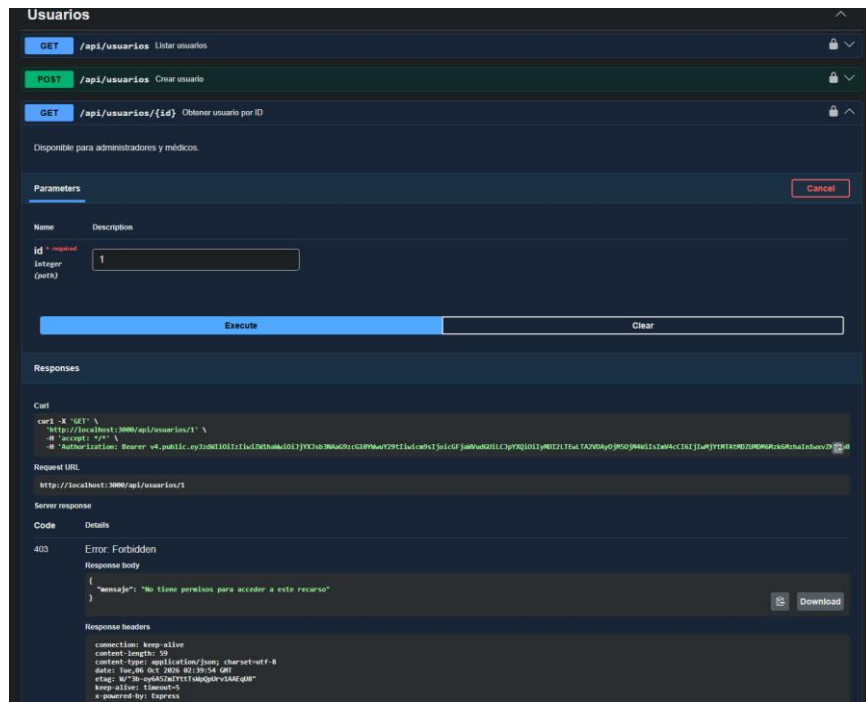


Figura 43. Prueba 21c. Paciente consultando el usuario 1: 403.

Credenciales incorrectas y usuario deshabilitado

Con una contraseña incorrecta (prueba 22) y con un correo inexistente (prueba 23) el login responde el mismo 401, para que nadie pueda usarlo y descubrir qué correos están registrados. Como prueba adicional, el usuario que desactivé con el **PATCH** ya no puede iniciar sesión y recibe 403.

Usuarios

GET

/api/usuarios

Listar usuarios

POST

/api/usuarios

Crear usuario

GET

/api/usuarios/{id}

Obtener usuario por ID

PUT

/api/usuarios/{id}

Reemplazar completamente un usuario

PATCH

/api/usuarios/{id}

Actualizar parcialmente un usuario

DELETE

/api/usuarios/{id}

Eliminar usuario

Autenticación

POST

/api/auth/login

Iniciar sesión

Valida las credenciales y genera un PASSETO v4 public.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "email": "admin@hospital.com",
  "password": "incorrecta"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3000/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "admin@hospital.com",
    "password": "incorrecta"
  }'
```

Request URL

http://localhost:3000/api/auth/login

Server response

Code

Details

401

Error: Unauthorized

Response body

```
{
  "mensaje": "Credenciales inválidas"
}
```

Download

Response headers

```
connection: keep-alive
content-length: 37
content-type: application/json; charset=utf-8
date: Tue, 06 Oct 2020 02:39:58 GMT
etag: W/"25-6d1d0e7d6079a4351-830"
keep-alive: timeout=5
powered-by: Express
```

Figura 44. Prueba 22. Contraseña incorrecta: 401.

Usuarios

GET

/api/usuarios

Listar usuarios

POST

/api/usuarios

Crear usuario

GET

/api/usuarios/{id}

Obtener usuario por ID

PUT

/api/usuarios/{id}

Reemplazar completamente un usuario

PATCH

/api/usuarios/{id}

Actualizar parcialmente un usuario

DELETE

/api/usuarios/{id}

Eliminar usuario

Autenticación

POST

/api/auth/login

Iniciar sesión

Valida las credenciales y genera un PASSETO v4 public.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{  "email": "noexistehospital.com",  "password": "123456"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  "http://localhost:3000/api/auth/login" \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "email": "noexistehospital.com",  "password": "123456"  }'
```

Request URL

http://localhost:3000/api/auth/login

Server response

Code

Details

401

Error: Unauthorized

Response body

```
{  "mensaje": "Credenciales inválidas"}
```

Download

Response headers

```
connection: keep-alive  content-length: 37  content-type: application/json; charset=utf-8  date: Tue, 26 Oct 2026 02:40:02 GMT  etag: W/"25-611d0e72a07079a7351-830"  keep-alive: timeout=5  x-powered-by: Express
```

Figura 45. Prueba 23. Correo inexistente: 401.

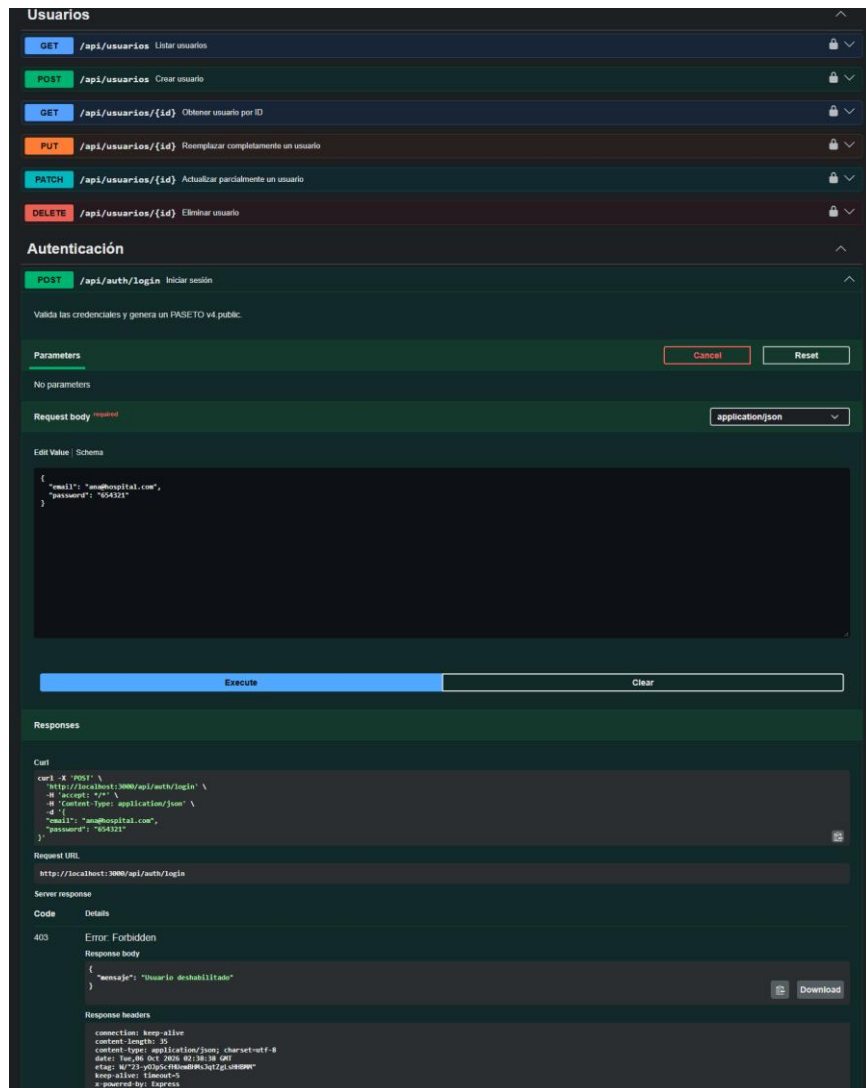


Figura 46. Prueba adicional. Login de un usuario deshabilitado: 403.

8. Lo que estos laboratorios no resuelven

Los laboratorios están simplificados a propósito y hay cosas que no usaría en producción. Las contraseñas están en texto plano y deberían guardarse con un hash como bcrypt o Argon2. Los usuarios viven en memoria y se pierden al reiniciar el servidor. Las claves PASETO se generan en cada arranque, así que no hay forma de mantener sesiones entre reinicios, y deberían guardarse y rotarse de forma segura.

Tampoco hay forma de revocar un token antes de que venza, y faltan cosas como HTTPS, rate limiting, Helmet, CORS restringido, registro de eventos y pruebas automatizadas.

9. Resumen

La V1 resolvió el login y la verificación del token, con 5 pruebas. La V2 sumó usuarios, roles, la diferencia entre 401 y 403, Swagger y el análisis con `npm audit`, con 12 pruebas. La V3 agregó el CRUD completo con validaciones y `matchedData()`, con 23 pruebas y una adicional. Lo más importante que me quedó es que la seguridad de la API no depende solo del token: PASETO dice

quién eres, los roles definen qué puedes hacer y las validaciones controlan lo que entra, y las tres capas tienen que ir en ese orden.

10. Conclusiones

Con los tres laboratorios logré implementar una API protegida con PASETO v4.public, que firma el token con una clave privada y lo verifica con una clave pública, y comprobé con pruebas reales que un token alterado, ausente o sin el rol adecuado no pasa. También entendí que un hallazgo de `npm audit` hay que analizarlo antes de corregirlo, porque el que apareció venía de una herramienta de desarrollo.

Aun así, la API es de laboratorio. Para llevarla a producción faltaría guardar las contraseñas con hash, usar una base de datos y manejar las claves de forma segura y persistente.